

Advanced NumPy

Data Science Mentorship Program | Verified Code & Outputs

1. NumPy Arrays vs. Python Lists

NumPy arrays are preferred over standard Python lists for data science tasks due to three main reasons:

- **Speed of Execution:** NumPy is approximately 50x faster than standard Python lists because it uses static, contiguous C-type arrays.
- **Memory Efficiency:** Python lists consume vastly more RAM. NumPy allows specifying optimized data types (e.g., `int32`, `int16`).
- **Convenience:** Mathematical operations are automatically vectorized.

2. Advanced Indexing

Fancy Indexing

Extract specific rows or columns passing a list of index positions.

```
import numpy as np

arr = np.arange(1, 17).reshape(4, 4)
# arr is:
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]
#  [13 14 15 16]]

# Extracting the 1st, 3rd, and 4th row (indices 0, 2, 3)
print(arr[[0, 2, 3]])
```

OUTPUT

```
[[ 1  2  3  4]
 [ 9 10 11 12]
 [13 14 15 16]]
```

```
# Extracting all rows, but only the 1st, 2nd, and 4th column (indices 0, 1, 3)
print(arr[:, [0, 1, 3]])
```

OUTPUT

```
[[ 1  2  4]
 [ 5  6  8]
 [ 9 10 12]
 [13 14 16]]
```

Boolean Indexing (Boolean Masking)

Filter items based on logical conditions.

```
arr = np.array([45, 55, 60, 49, 14])

# 1. Filter items greater than 50
print(arr[arr > 50])
```

OUTPUT

```
[55 60]
```

```
# 2. Chaining multiple conditions (e.g., > 50 AND even number)
print(arr[(arr > 50) & (arr % 2 == 0)])
```

OUTPUT

```
[60]
```

```
# 3. Filter out numbers divisible by 7 (using the NOT operator ~)
print(arr[~(arr % 7 == 0)])
```

OUTPUT

```
[45 55 60]
```

3. Broadcasting

Broadcasting automatically "stretches" the smaller array so their shapes match during arithmetic operations without copying memory.

```
# Array A has shape (2, 3)
A = np.array([[1, 2, 3],
              [4, 5, 6]])

# Array B has shape (3,)
B = np.array([10, 20, 30])

# B is broadcasted to [[10, 20, 30], [10, 20, 30]] implicitly
print(A + B)
```

OUTPUT

```
[[11 22 33]
 [14 25 36]]
```

4. Custom Mathematical Functions

You can write standard Python functions and NumPy handles the vectorized execution across all items instantly.

Example 1: Sigmoid Function

```
def sigmoid(array):
    return 1 / (1 + np.exp(-array))

arr = np.array([0, 1, 2])
print(sigmoid(arr))
```

OUTPUT

```
[0.5      0.73105858 0.88079708]
```

Example 2: Mean Squared Error (MSE)

```
def mse(actual, predicted):
    return np.mean((actual - predicted)**2)

actual_data = np.array([10, 20, 30])
predicted_data = np.array([12, 19, 30])

print(mse(actual_data, predicted_data))
```

OUTPUT

```
1.6666666666666667
```

Calculation: $((10-12)^2 + (20-19)^2 + (30-30)^2)/3 = (4 + 1 + 0)/3 = 5/3 \approx 1.666...$

5. Dealing with Missing Values (`np.nan`)

NumPy uses `np.nan` to represent missing data. You can filter them out using `~np.isnan()`.

```
arr = np.array([1, 2, np.nan, 4, np.nan])

# Create a clean array without NaNs
clean_array = arr[~np.isnan(arr)]
print(clean_array)
```

OUTPUT

```
[1. 2. 4.]
```

6. Plotting Mathematical Equations

Using `np.linspace` and `matplotlib.pyplot`, you can visualize any mathematical equation quickly.

```
import matplotlib.pyplot as plt
import numpy as np

# Generate 100 points between -10 and 10
x = np.linspace(-10, 10, 100)

y = x**2
plt.plot(x, y)
plt.show()
```

OUTPUT (VISUAL)

A 2D line plot showing a parabolic curve opening upwards, with the vertex at (0,0), passing through (-10, 100) and (10, 100).